Python Activity-Based Learning

Case Study 13: ATM Transaction Sequence Simulation

1. Problem Statement

Simulate a sequence of ATM deposits and withdrawals, maintain the account balance, enforce transaction rules, reject invalid transactions, and summarize accepted transactions.

2. Objectives

- Use a loop to process multiple transactions.
- Use conditional statements to validate deposits and withdrawals.
- Maintain balance and count successful transactions.

3. Assumptions

- Opening balance is ₹5,000.
- Each withdrawal must be positive, cannot exceed the balance, and cannot exceed ₹10,000.
- Deposits must be positive.
- Only valid transactions change the balance.

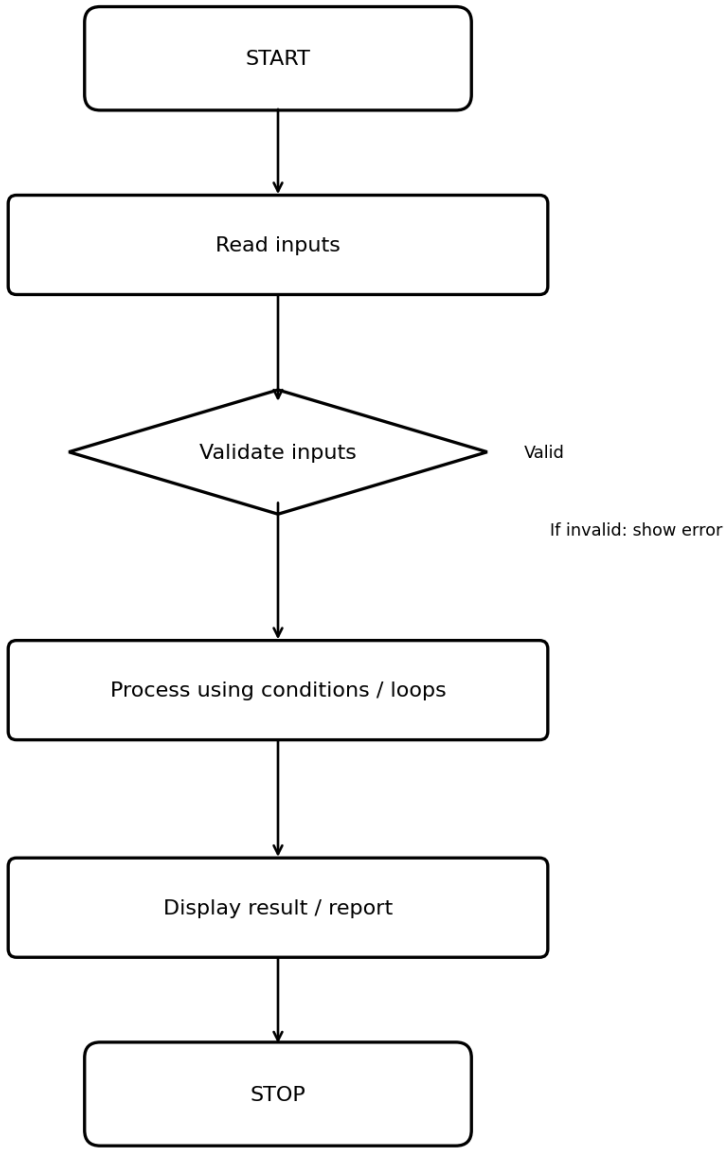
4. Algorithm

1. Set the opening balance and transaction counters.
2. Read the number of transactions.
3. For each transaction, read type and amount.
4. Validate the amount and withdrawal limits.
5. Update the balance and counters only for valid transactions.
6. Display transaction result and final summary.

5. Flowchart

The diagram below shows the overall program flow. The validation decision represents the input checks described in the algorithm.

Flowchart: ATM Transaction Sequence Simulation



6. Pseudocode

```
START
balance = 5000
successful = 0
READ transaction_count
FOR each transaction
    READ transaction_type, amount
    IF amount <= 0 THEN DISPLAY invalid amount
    ELSE IF type is deposit THEN
        balance = balance + amount
        successful = successful + 1
    ELSE IF type is withdrawal THEN
        IF amount > 10000 OR amount > balance THEN DISPLAY rejected
        ELSE balance = balance - amount; successful = successful + 1
    ELSE DISPLAY unknown transaction type
```

```
    END IF
END FOR
DISPLAY final balance and successful count
STOP
```

7. Python Source Code

The solution uses only basic Python features from the assigned syllabus.

```
balance = 5000.0
successful = 0
deposits = 0.0
withdrawals = 0.0
count = int(input("Number of transactions: "))

for i in range(count):
    kind = input("Transaction type (deposit/withdraw): ").lower()
    amount = float(input("Amount (₹): "))

    if amount <= 0:
        print("Rejected: amount must be positive.")
    elif kind == "deposit":
        balance += amount
        deposits += amount
        successful += 1
        print(f"Deposit accepted. Balance: ₹{balance:.2f}")
    elif kind == "withdraw":
        if amount > 10000:
            print("Rejected: withdrawal limit is ₹10000.")
        elif amount > balance:
            print("Rejected: insufficient balance.")
        else:
            balance -= amount
            withdrawals += amount
            successful += 1
            print(f"Withdrawal accepted. Balance: ₹{balance:.2f}")
    else:
        print("Rejected: unknown transaction type.")

print("\n------ TRANSACTION SUMMARY ------")
print(f"Opening balance : ₹5000.00")
print(f"Total deposits  : ₹{deposits:.2f}")
print(f"Total withdrawals: ₹{withdrawals:.2f}")
print(f"Successful transactions: {successful}")
print(f"Closing balance : ₹{balance:.2f}")
```

8. Test Cases

Test input / condition	Expected result	Purpose
deposit 1000	₹1,000 deposit	Balance becomes ₹6,000
withdraw 7000	Opening balance ₹5,000	Rejected for insufficient balance
withdraw 12000	Any balance	Rejected because amount exceeds limit

9. Sample Output Screenshot

Sample console output for one test run:

```
C:\Users\Badri\PycharmProjects\PythonProject-1\.venv\Scripts\python.exe "C:\Users\Badri\PycharmProjects\PythonProject-1\Case Studies\case
 study - 8.py"
Number of transactions: 1
Transaction type (deposit/withdraw): withdraw
Amount (₹): 1000
Withdrawal accepted. Balance: ₹4000.00

------ TRANSACTION SUMMARY ------
Opening balance : ₹5000.00
Total deposits  : ₹0.00
Total withdrawals: ₹1000.00
Successful transactions: 1
Closing balance : ₹4000.00

Process finished with exit code 0
```

10. Conclusion

The simulation demonstrates sequence processing, validation, conditional logic, and accumulation of transaction totals.

Note: Assumed rates, thresholds, and sample values are for this educational exercise and can be changed if the instructor supplies different rules.